

SET UP Jetson AGX Xavier X221-AI

Jetson AGX Xavier Developer Kit 16GB (NVIDIA Xavier SoC)

Board: X221 <https://auvidea.eu/product/x221-70415/>

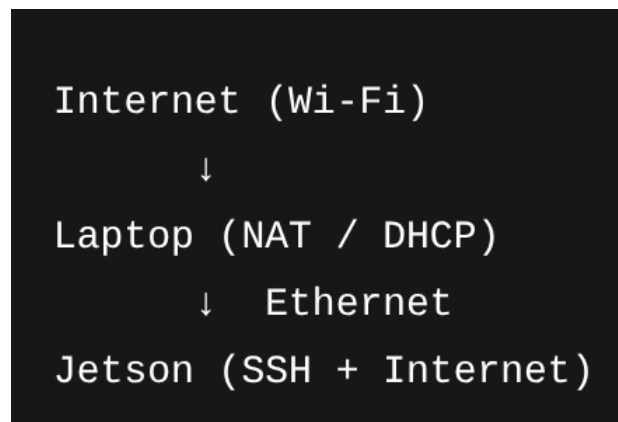
X221_Manual https://auvidea.eu/download/X221_Manual_v2.0.pdf

1. OS: Ubuntu 20.04

2. SSH

```
ssh nguyenan@192.168.3.71
Password : 1
```

▼ LAN trực tiếp (có wifi)



Điều kiện

- Laptop có **2 kết nối mạng cùng lúc**
 - Wi-Fi (ra Internet)
 - LAN RJ45 (nối Jetson)
- Jetson để **DHCP auto** (không IP tĩnh cứng)
- Có màn hình cho Jetson để check IP (mỗi lần mỗi khác)

Bước 1 – Cắm LAN Laptop ↔ Jetson

Bước 2 – Mở Network Settings

Bước 3 – Chỉnh Wired (Ethernet)

Nhấn biểu tượng bánh răng của **Wired** → tab **IPv4**

- Method: **Shared to other computers**
- Apply.
- OFF → ON **Wired Setting laptop + jetson**

Ngay lập tức Ubuntu sẽ:

- Tạo mạng `10.42.0.0/24`
- Laptop LAN: `10.42.0.1`
- Jetson sẽ tự nhận `10.42.0.x`
- Bật NAT + DHCP nội bộ

Bước 4 – Kiểm tra Jetson

Trên Jetson:

```
ip a
```

Bạn sẽ thấy:

```
inet10.42.0.122/24
```

Bước 5 – SSH

Trên Laptop:

```
ssh -Y nguyenan@10.42.0.122
```

Jetson lúc này:

- Có SSH
- Có Internet
- Không cần IP tĩnh
- Không cần router
- Có thể hiển thị GUI của Jetson trên Laptop.

▼ LAN trực tiếp (không wifi)

Cấu hình IP tĩnh vĩnh viễn trên Jetson

Dùng **NetworkManager**

Bước 1 — Xác định tên cổng LAN

Trên Jetson:

```
ip a
```

Tìm interface Ethernet, có thể là `enp2s0`.

1. Xem tên kết nối

```
nmcli connection show
```

Bạn sẽ thấy kiểu như:

```
Wired connection1
```

2. Sửa IP tĩnh

```
sudo nmcli connection modify "Wired connection 1" \
ipv4.method manual \
ipv4.addresses 192.168.77.2/24
```

3. Khởi động lại kết nối

```
sudo nmcli connection down "Wired connection 1"
sudo nmcli connection up "Wired connection 1"
```

Cấu hình Laptop (mỗi khi LAN trực tiếp)

Chỉ cần đặt IP cùng subnet, ví dụ:

```
192.168.77.1
255.255.255.0
Gateway:trống
```

Sau đó SSH:

```
ssh -Y nguyenan@192.168.77.2
```

▼ LAN qua Router

▼ Wifi

1. Cài đặt OpenSSH Server:

Mở Terminal trên Jetson và chạy lệnh: Bash

```
sudo apt update
sudo apt install openssh-server
```

2. Kiểm tra trạng thái:

Đảm bảo dịch vụ đang chạy: Bash

```
sudo systemctl status ssh
```

(Nếu thấy chữ "active (running)" màu xanh là ổn).

3. Xác định địa chỉ IP của Jetson:

Gõ lệnh sau để lấy IP Wi-Fi (thường bắt đầu bằng 192.168...): Bash

```
hostname -I
```

Lưu ý: Cả laptop và Jetson phải kết nối cùng một mạng Wi-Fi.

Bước 2: Kết nối từ Laptop

Bây giờ, hãy quay lại laptop Ubuntu của bạn.

1. Mở Terminal (Ctrl + Alt + T).

2. Sử dụng lệnh SSH:

Cấu trúc lệnh: `ssh username@dia_chi_ip`

```
ssh -Y nguyenan@192.168.1.20
```

3. ROS 2 Foxy

▼ Cài đặt

1. Kiểm tra hệ điều hành và locale

1.1. Kiểm tra Ubuntu version

```
lsb_release -a
```

Kết quả phải là:

```
Ubuntu 20.04 LTS
```

1.2. Thiết lập locale (bắt buộc cho ROS 2)

```
sudo apt update
sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Kiểm tra:

```
locale
```

2. Thêm ROS 2 apt repository

2.1. Cài các gói cần thiết

```
sudo apt install software-properties-common curl
```

2.2. Thêm ROS 2 GPG key

```
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.key | sudo apt-key add -
```

2.3. Thêm repository ROS 2 Foxy

```
sudo add-apt-repository universe
sudo apt update
```

Thêm source list:

```
sudo sh -c 'echo "deb http://packages.ros.org/ros2/ubuntu focal main" > /etc/apt/sources.list.d/ros2.list'
```

Update lại:

```
sudo apt update
```

3. Cài đặt ROS 2 Foxy

3.1. Cách 1 (Khuyến dùng): ROS 2 Desktop

Bao gồm:

- RViz2
- demo nodes
- tools
- visualization

```
sudo apt install ros-foxy-desktop
```

3.2. Cách 2: Minimal (chỉ core)

```
sudo apt install ros-foxy-ros-base
```

🔴 Với **robot + LiDAR + navigation**, bạn nên dùng **desktop**.

4. Thiết lập môi trường ROS 2

4.1. Source ROS 2 mỗi lần mở terminal

```
source /opt/ros/foxy/setup.bash
```

4.2. Tự động source khi mở terminal

```
echo "source /opt/ros/foxy/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

Kiểm tra:

```
printenv | grep ROS
```

5. Cài công cụ phát triển (rất quan trọng)

```
sudo apt install-y \  
python3-colcon-common-extensions \  
python3-rosdep \  
python3-argcomplete
```

5.1. Khởi tạo rosdep

```
sudo rosdep init  
rosdep update
```

▼ Kiểm tra package ROS 2 Foxy

Kiểm tra package ROS 2 Foxy đã được cài hay chưa (xác nhận ở mức hệ thống)

```
dpkg-l |grep ros-foxy
```

Kết quả đúng

- Thấy các dòng như:

```
ii ros-foxy-desktop ...  
ii ros-foxy-rclcpp ...  
ii ros-foxy-ros-base ...
```

✓ Nghĩa là ROS 2 Foxy **đã được cài thật sự**, không chỉ thêm repository.

2. Kiểm tra thư mục cài đặt chuẩn của ROS 2

```
ls /opt/ros
```

3. Kiểm tra ROS 2 đã hoạt động chưa

3.1. Mở terminal 1

```
ros2 run demo_nodes_cpp talker
```

3.2. Mở terminal 2

```
ros2 run demo_nodes_py listener
```

Nếu thấy message truyền qua lại → **ROS 2 Foxy OK** ✓

▼ Cài Navigation2 Foxy

```
sudo apt update
sudo apt install ros-foxy-navigation2 ros-foxy-nav2-bringup
```

Sau đó:

```
source /opt/ros/foxy/setup.bash
```

Kiểm tra:

```
ros2 pkg list | grep nav2
```

Bạn phải thấy:

```
nav2_bringup
nav2_controller
nav2_planner
nav2_bt_navigator
...
```

▼ Cài slam_toolbox

Cài slam_toolbox cho ROS 2 Foxy

```
sudo apt update
sudo apt install ros-foxy-slam-toolbox
```

Kiểm tra:

```
ros2 pkg list | grep slam
```

Phải thấy:

slam_toolbox

Bước 1: Cài đặt gói **xacro**

Mở terminal và chạy lệnh sau để cài đặt **xacro** cho ROS 2 Foxy:

Bash

```
sudo apt update
sudo apt install ros-foxy-xacro
```

Bước 2: Cài đặt đầy đủ các dependencies (Khuyến dùng)

Mặc dù trong file `package.xml` bạn đã khai báo `<exec_depend>xacro</exec_depend>`, nhưng có vẻ bạn chưa chạy lệnh cài đặt dependency tự động. Để tránh thiếu các gói khác (như `gazebo_ros`, `nav2_bringup` ...), hãy chạy lệnh chuẩn sau tại thư mục gốc của workspace (`~/ros2_ws`):

Bash

```
# Tại thư mục ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
```

Lệnh này sẽ tự động đọc `package.xml` và cài tất cả các thư viện còn thiếu.

Bước 1: Cài đặt gói còn thiếu

Mở terminal và chạy lệnh:

Bash

```
sudo apt update
sudo apt install ros-foxy-joint-state-publisher
```

(Nếu bạn muốn dùng giao diện điều chỉnh khớp robot thủ công, có thể cài thêm `ros-foxy-joint-state-publisher-gui`, nhưng với file launch hiện tại thì lệnh trên là đủ).

Bước 2: Cài đặt TẤT CẢ các thư viện phụ thuộc (Rất quan trọng)

Để tránh việc cứ chạy lại báo thiếu một gói khác (ví dụ tiếp theo có thể thiếu `gazebo_ros`, `nav2_bringup`, v.v.), bạn hãy chạy lệnh sau để ROS tự động quét file `package.xml` và cài hết tất cả những gì còn thiếu một lần duy nhất:

Bash

```
cd ~/ros2_ws
# Lệnh này sẽ tự động cài các gói exec_depend còn thiếu trong package.xml
rosdep install --from-paths src --ignore-src -r -y
```

Cài đặt `robot-localization`

Bạn cần mở Terminal trên Jetson AGX Xavier (hoặc qua SSH) và chạy lệnh sau để cài đặt gói EKF (Extended Kalman Filter):

Bash

```
sudo apt update
sudo apt install ros-foxy-robot-localization
```

2. Cài đặt tự động các dependencies (Khuyên dùng)

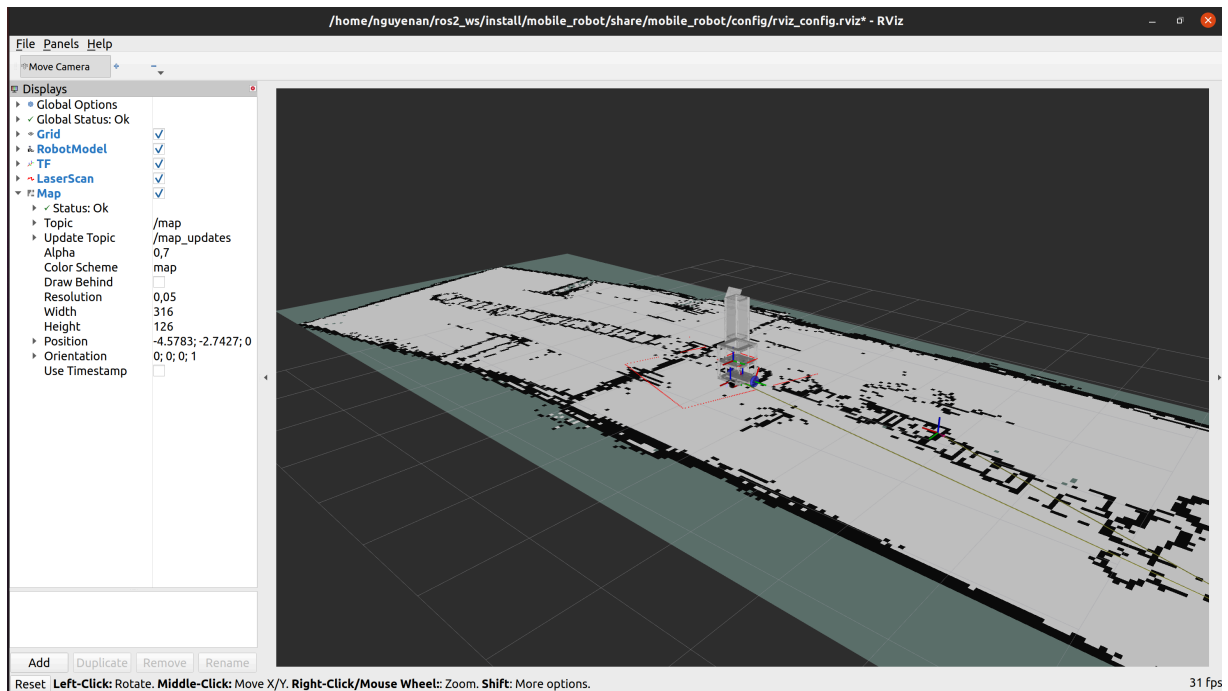
Để tránh gặp lỗi tương tự với các gói khác (như `slam_toolbox`, `navigation2`, `xacro` ...), bạn nên sử dụng `rosdep` để quét và cài đặt tất cả các thư viện còn thiếu dựa trên file `package.xml` của bạn.

Tại thư mục gốc của workspace (`~/ros2_ws`), chạy lệnh:

Bash

```
# Cập nhật cơ sở dữ liệu rosdep
sudo rosdep init
rosdep update

# Cài đặt tất cả dependencies
rosdep install --from-paths src --ignore-src -r -y
```



3. Cài đặt Laser filters

```
sudo apt update
```

```
sudo apt install ros-foxy-laser-filters
```

Sau đó:

```
source /opt/ros/foxy/setup.bash
```

Kiểm tra:

```
ros2 pkg list |grep laser_filters
```

Nếu thấy:

```
laser_filters
```

→ OK.

▼ LiDAR

```
ip addr
sudo ip addr add 192.168.11.1/24 dev enp2s0
sudo ip link set enp2s0 up
```

Sau khi chạy xong, bạn kiểm tra lại bằng `ip addr`. Nếu thấy dòng `inet 192.168.11.1/24 ...` xuất hiện ở mục số 4 (`enp2s0`) là thành công.

Bước 2: Kiểm tra kết nối tới LiDAR (Ping)

```
ping 192.168.11.2
```

```
# Cài sllidar_ros2 (hỗ trợ S2E, khác với rplidar_ros cũ)
cd ~/mbrobot_ws/src
git clone https://github.com/Slamtec/sllidar_ros2.git

cd ~/mbrobot_ws
colcon build --packages-select sllidar_ros2 --symlink-install
source install/setup.bash

# Chạy thư'file launch có sẵn
ros2 launch sllidar_ros2 view_sllidar_s2e_launch.py

# Kiểm tra port đúng từ launch file
cat $(ros2 pkg prefix sllidar_ros2)/share/sllidar_ros2/launch/sllidar_s2e_launch.py |
grep tcp

# Xem frame_id mà LiDAR đang publish
# Terminal 1:
ros2 topic echo /scan --no-arr | grep frame_id

# Terminal 2:
ros2 run sllidar_ros2 sllidar_node \
  --ros-args \
  -p channel_type:=udp \
  -p udp_ip:=192.168.11.2 \
  -p udp_port:=8089 \
  -p frame_id:=laser_frame \
  -p angle_compensate:=true \
  -p scan_mode:=Sensitivity
```

4. Kiểm tra xem dữ liệu đã về chưa

Mở một terminal mới (SSH vào Jetson) và gõ:

```
ros2 topic list
```

Nếu thấy xuất hiện `/scan`, hãy kiểm tra dữ liệu thực tế:

```
ros2 topic echo /scan
```

▼ IMU

Cài đặt Package

Bước 1 — Cài lại `bno055` (flynnova/bno055)

Đây là phần **suy luận có cơ sở, không phải trích từ tài liệu dự án** — bạn đã đồng ý dùng repo này.

```
cd ~/mbroot_ws/src
rm -rf bno055 # xoá bản cũ nếu còn sót
git clone https://github.com/flynnova/bno055.git
```

```
cd ~/mbroot_ws
colcon build --packages-select bno055 --symlink-install
source install/setup.bash
```

Bước 3 — Rebuild lại `mobile_robot` để đồng bộ

```
cd ~/mbroot_ws
colcon build --packages-select mobile_robot --symlink-install
source install/setup.bash
```

Sơ đồ kết nối phần cứng

BN0055	J23 (JST-GH 1.25mm)
VIN (3.3V)	Pin 1 (3.3V)
GND	Pin 8 (GND)
SCL	Pin 6 (I2C_SCL_GP5_3.3)
SDA	Pin 7 (I2C_SDA_GP5_3.3)
ADR	GND (→ địa chỉ I2C = 0x28)
	hoặc
ADR	3.3V (→ địa chỉ I2C = 0x29)

Không cần thêm pull-up resistor vì I2C GP5 trên Jetson AGX Xavier đã có internal pull-up.

Xác định I2C Bus Number trên Jetson

I2C_GP5 (socket pin A53/C53) trên Jetson AGX Xavier tương ứng với `/dev/i2c-8`.

Bước 1: Cài i2c tools trên Jetson:

```
sudo apt-get install -y i2c-tools
```

Bước 2: Liệt kê tất cả I2C buses:

```
ls /dev/i2c-*
# Kết quả thường thấy: i2c-0, i2c-1, i2c-2, i2c-4, i2c-7, i2c-8
```

Bước 3: Scan BNO055 trên bus 8:

```
sudo i2cdetect -y -r 8

# Kết quả mong đợi (ADR nối GND):
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- 28 -- -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

Nếu không thấy `0x28`, thử lần lượt với `-y 0`, `-y 1`, `-y 7` để xác nhận đúng bus.

Bước 4: Kiểm tra Chip ID của BNO055 (register 0x00 = 0xA0):

```
sudo i2cget -y 8 0x28 0x00
# Output: 0xa0 ← đây là BNO055 Chip ID
```

Phân quyền I2C

Kiểm tra quyền mặc định

```
ls -la /dev/i2c-*
```

Kết quả thường thấy:

```
crw-rw---- 1 root i2c 89, 0 Mar 11 10:00 /dev/i2c-0
crw-rw---- 1 root i2c 89, 8 Mar 11 10:00 /dev/i2c-8
```

Phân tích cấu trúc quyền `crw-rw----`:

```
c rw- rw- ---
| | |
| | | — other: không có quyền gì
| | | — group: read + write ← group "i2c"
| | | — owner: read + write ← root
| | — c = character device
```

****Kết luận:**** Chỉ `root` và các user thuộc **group `i2c`** mới đọc/ghi được I2C. Nếu không phân quyền, mọi lệnh `i2cdetect`, `i2cget`, hay ROS node sẽ báo:

Error: Could not open file '/dev/i2c-8': Permission denied

Kiểm tra trạng thái hiện tại

Bước 1: Kiểm tra group `i2c` có tồn tại chưa:

```
getent group i2c
```

- Nếu **có kết quả** (ví dụ `i2c:x:998:`) → group đã tồn tại.
- Nếu **không có kết quả** → cần tạo group (xem bước 3.1).

Bước 2: Kiểm tra user hiện tại thuộc những group nào:

```
groups $USER
# Ví dụ: nvidia adm sudo dialout i2c
```

Nếu `i2c` **không có** trong danh sách → cần thêm user vào group.

Bước 3: Kiểm tra quyền file I2C cụ thể:

```
ls -la /dev/i2c-8
stat /dev/i2c-8
```

Bước 3.3 — Tạo udev rule để quyền tự động áp dụng khi reboot

Vấn đề: Quyền trên `/dev/i2c-*` bị reset mỗi lần boot vì `/dev` là tmpfs (filesystem ảo trong RAM). `udev` là daemon quản lý device files — ta cần tạo rule để nó tự động gán đúng group và permission khi hệ thống phát hiện thiết bị I2C.

```
# Tạo udev rule file
sudo nano /etc/udev/rules.d/99-i2c.rules

# Nội dung file:
KERNEL=="i2c-[0-9]*", GROUP="i2c", MODE="0660"
```

Giải thích từng phần của rule:

Phần	Ý nghĩa
<code>KERNEL=="i2c-[0-9]*"</code>	Match tất cả device có tên dạng <code>i2c-0</code> , <code>i2c-8</code> , ...
<code>GROUP="i2c"</code>	Gán group owner là <code>i2c</code>
<code>MODE="0660"</code>	Quyền: owner=rw, group=rw, other=none

Reload và áp dụng rule ngay (không cần reboot):

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Xác nhận rule đã có hiệu lực:

```
ls -la /dev/i2c-8
# crw-rw---- 1 root i2c 89, 8 Mar 11 10:00 /dev/i2c-8
#                ^^^ group phải là "i2c"
```

Kiểm tra kết quả

```
ros2 pkg list | grep -E "sllidar_ros2|^bno055"
```

Chạy thử IMU (nguồn: [README.md](#), mục "IMU"):

```
ros2 run bno055 bno055 \
  --ros-args \
  --params-file ~/mbroot_ws/src/mobile_robot/config/bno055_params.yaml \
  --remap /imu/imu:=/imu/data

# Terminal khác:
ros2 topic list | grep imu
ros2 topic hz /imu/data
```

Nếu `/dev/i2c-8` báo `Permission denied`, quay lại mục "Phân quyền I2C" trong [SET_UP_Jetson_AGX_Xavier_X221-AI.md](#) (dòng 445–534) — bạn đã có sẵn udev rule cho `i2c` theo tài liệu đó.

Cài rviz_imu_plugin

Bước 1 — Cài plugin trên Laptop (nơi chạy RViz)

```
sudo apt update
sudo apt install ros-foxy-rviz-imu-plugin
```

Kiểm tra cài thành công:

```
ros2 pkg list | grep imu
# → rviz_imu_plugin
```

▼ STM32 (Hover Board) ↔ Jetson

STM32: STM32F103RCT6.

Firmware STM32: hoverboard-firmware-hack-FOC (link: <https://github.com/EFeru/hoverboard-firmware-hack-FOC?tab=readme-ov-file>).

Jetson/X221-AI (Port J33) — USB-to-TTL (UART) — STM32.

Tần số gửi feedback từ STM32: ... Hz

▼ Giao tiếp STM32

Kiểm tra quyền hiện tại

```
ls -la /dev/ttyUSB0
```

crw-rw---- 1 root dialout 188, 0 ...

Thêm user vào group dialout

```
sudo usermod -aG dialout $USER
```

Áp dụng ngay mà không cần logout

```
newgrp dialout
```

Verify

```
groups $USER
```

Kết quả phải có: dialout

▼ rviz_laptop

Cấu hình DDS Unicast

Bước 1 — Tạo file XML cấu hình DDS trên cả laptop lẫn Jetson

Trên Laptop (~/fastdds_laptop.xml):

xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<profiles xmlns="http://www.eprosima.com/XMLSchemas/fastRTPS_Profiles">
  <participant profile_name="unicast_participant" is_default_profile="true">
    <rtps>
      <builtin>
        <metatrafficUnicastLocatorList>
          <locator>
            <udp4>
              <address>192.168.3.70</address>
            </udp4>
          </locator>
        </metatrafficUnicastLocatorList>
        <initialPeersList>
          <locator>
            <udp4>
              <address>192.168.3.71</address>
            </udp4>
          </locator>
        </initialPeersList>
        <use_WriterLivelinessProtocol>false</use_WriterLivelinessProtocol>
      </builtin>
    </rtps>
  </participant>
</profiles>
```

Trên Jetson (~/fastdds_jetson.xml):

xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<profiles xmlns="http://www.eprosima.com/XMLSchemas/fastRTPS_Profiles">
  <participant profile_name="unicast_participant" is_default_profile="true">
    <rtps>
      <builtin>
        <metatrafficUnicastLocatorList>
          <locator>
            <udp4>
              <address>192.168.3.71</address>
            </udp4>
          </locator>
        </metatrafficUnicastLocatorList>
        <initialPeersList>
          <locator>
            <udp4>
              <address>192.168.3.70</address>
            </udp4>
          </locator>
        </initialPeersList>
        <use_WriterLivelinessProtocol>false</use_WriterLivelinessProtocol>
      </builtin>
    </rtps>
  </participant>
</profiles>
```

Bước 2 — Set biến môi trường

Thêm vào `~/bashrc` trên **Laptop**:

bash

```
export ROS_DOMAIN_ID=42
export ROS_LOCALHOST_ONLY=0
export FASTRTPS_DEFAULT_PROFILES_FILE=~/fastdds_laptop.xml
```

Thêm vào `~/bashrc` trên **Jetson**:

bash

```
export ROS_DOMAIN_ID=42
export ROS_LOCALHOST_ONLY=0
export FASTRTPS_DEFAULT_PROFILES_FILE=~/fastdds_jetson.xml
```

Sau đó:

bash

```
# Cả hai máy
source ~/.bashrc
```

Bước 3 — Mở firewall (nếu đang bật)

Chạy trên **cả hai máy**:

bash

```
sudo ufw allow in proto udp from 192.168.3.0/24
sudo ufw allow out proto udp to 192.168.3.0/24
```

Bước 4 — Kiểm tra kết nối

Trên Jetson — chạy `view_lidar_launch`:

bash

```
ros2 launch mobile_robot view_lidar_launch.py
```

Trên Laptop — verify topics thấy từ Jetson:

bash

```
ros2 topic list
ros2 topic echo /scan --once
```

Nếu thấy `/scan` và `/tf` → chạy RViz:

bash

```
ros2 launch mobile_robot rviz_launch.py
```

▼ App

```
git pull
make setup
make clean
make build
make run
```

▼ File kiểm tra phần cứng

```
# Copy vào workspace
cp preflight_check.sh ~/mbrobot_ws/src/mobile_robot/scripts/
chmod +x ~/mbrobot_ws/src/mobile_robot/scripts/preflight_check.sh

# Tạo alias
echo 'alias preflight="bash ~/mbrobot_ws/src/mobile_robot/scripts/preflight_check.sh"'
>> ~/.bashrc
source ~/.bashrc

# Chạy
preflight
preflight --motor-test
```

tyjyg

▼ Config UDEV rule

```
sudo nano /etc/udev/rules.d/99-robot-serial.rules

# ESP32 ultrasonic - cong USB 3.0 type-A (root hub port 2)
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", KERNELS=="1-2",
SYMLINK+="ttyUltrasonic"
# STM32 hoverboard - J33 = USB(2) = hub Genesys port 2
SUBSYSTEM=="tty", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", KERNELS=="1-3.2",
SYMLINK+="ttyWheel"
```

Save then reload:

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Verify:

```
ls -la /dev/ttyWheel /dev/ttyUltrasonic
```

▼ Thêm

```
sudo apt update
sudo apt install -y \
ros-foxy-slam-toolbox \
ros-foxy-robot-localization \
ros-foxy-laser-filters \
ros-foxy-robot-state-publisher \
ros-foxy-joint-state-publisher \
ros-foxy-xacro \
ros-foxy-tf2-ros \
ros-foxy-tf2-tools
```

Cài pyserial

```
sudo apt install python3-serial -y
```

Các gói thêm

▼ Driver usb wifi TPlink AC600

1. Cài đặt gói Kernel Headers của NVIDIA

Thay vì tìm gói theo tên phiên bản kernel, bạn hãy cài đặt gói tổng quát dành cho dòng L4T. Gói này sẽ tự động cung cấp các file cần thiết cho phiên bản kernel hiện tại của bạn.

Bash

```
sudo apt update
sudo apt install nvidia-l4t-kernel-headers
```

2. Kiểm tra xem file đã tồn tại chưa

Sau khi cài đặt xong, bạn hãy kiểm tra xem thư mục `build` (nơi driver sẽ tìm đến để biên dịch) đã trở đúng vào thư mục headers chưa:

Bash

```
ls -l /lib/modules/$(uname -r)/build
```

- **Nếu kết quả trả về một đường dẫn (thường là `/usr/src/linux-headers-...`):** Bạn đã thành công.
- **Nếu báo lỗi "No such file or directory":** Bạn cần tạo một liên kết biểu tượng (symlink) thủ công: Bash

```
sudo ln -s /usr/src/linux-headers-$(uname -r)-ubuntu20.04_aarch64/kernel-5.10 /lib/modules/$(uname -r)/build
```

3. Chạy lại lệnh cài đặt các công cụ còn lại

Bây giờ bạn có thể cài đặt các gói hỗ trợ khác mà không cần gọi lại gói `linux-headers` đã bị lỗi:

Bash

```
sudo apt install git dkms build-essential bc
```

4. Tải mã nguồn Driver RTL8821CU

Chúng ta sẽ sử dụng bản driver nguồn mở được bảo trì bởi `morrownr`. Bản này được cộng đồng đánh giá rất cao về tính ổn định trên kiến trúc ARM64 của Jetson:

Bash

```
git clone https://github.com/morrownr/8821cu-20210916.git
cd 8821cu-20210916
```

5. Tiến hành Biên dịch và Cài đặt

Chạy script cài đặt tự động. Script này sẽ sử dụng DKMS, lấy các file header mà bạn vừa kiểm tra ở trên để "hàn" driver vào đúng nhân L4T hiện tại:

Bash

```
sudo ./install-driver.sh
```

- **Lưu ý:** Trong quá trình chạy, terminal có thể sẽ hiện một câu hỏi (prompt) hỏi bạn có muốn mở và chỉnh sửa file cấu hình `8821cu.conf` hay không. Bạn chỉ cần chọn **"No"** (gõ `n` rồi nhấn Enter) để giữ nguyên cấu hình mặc định tối ưu. Quá trình biên dịch trên chip ARM có thể mất từ 1 đến 3 phút.

6. Khởi động lại hệ thống

Sau khi tiến trình cài đặt báo thành công, hãy đảm bảo USB Wi-Fi TP-Link Archer T2UB Nano đã được cắm chắc chắn vào cổng USB của Jetson, sau đó khởi động lại thiết bị để kernel nạp module mới:

Bash

```
sudo reboot
```

```
#
export ROS_DOMAIN_ID=42
export ROS_LOCALHOST_ONLY=0
export FASTRTPS_DEFAULT_PROFILES_FILE=~/.fastdds_laptop.xml
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp

# ----- CycloneDSS -----
```

```
# Jetson
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export ROS_DOMAIN_ID=42
export CYCLONEDDS_URI=file:///home/nguyenan/cyclone_jetson.xml

# Laptop
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export ROS_DOMAIN_ID=42
export CYCLONEDDS_URI=file:///home/nguyenan/cyclone_laptop.xml
```